

Lab1实验报告

李毅PB22051301

第一部分 实验原理

1.实验环境配置：

使用 WSL + QEMU 在 windows 下运行 linux 操作系统。

2.Multiboot 启动协议：

Multiboot 启动协议是一套多种操作系统共存时的启动引导协议。本次实验采用的协议版本为 Multiboot Specification version 0.6.96，实验目的为实现 Multiboot header 部分，并在 vga 屏幕和串口上输出特定内容。本次实验要求比较简单，我们只需实现最简单的 Multiboot header 格式要求即可。

3.VGA显存输出

本实验直接使用movl指令向 VGA 显存写入内容来实现字符输出。

4.UART串口输出

将串口端口地址存入 dx 寄存器，然后将要输出的字符的 ASCII 码存入 al 寄存器，最后调用 out 指令即可

第二部分 代码解释

- multibootHeader.S

```
1  .globl start # 一般都用start
2  /*此处，按照multiboot协议要求定义必要的参数*/
3  /*格式: XXX_ITEM_NAME=value*/
4  /*value可以是十六进制的（0x开头）、十进制的等等*/
5
6  # TODO 补全下面三个参数（直接参考ppt）
7  MAGIC_ITEM_NAME = 0x1BADB002
8  FLAGS_ITEM_NAME = 0x00000000
9  CHECKSUM_ITEM_NAME = -(MAGIC_ITEM_NAME + FLAGS_ITEM_NAME)
10
11 /*此处开始，按协议标准来定义必须的multiboot header*/
12
13 # TODO 将xxx自行修改为section的名字，利用定义好的参数，通过 .long 语句声明
Multiboot Header 的三个域
14 .section .multiboot
15 .align 4 #对齐！
16 /*使用.long和前面定义的参数构建头结构，每次32位，格式为: .long
XXX_ITEM_NAME*/
17     .long MAGIC_ITEM_NAME      # 待补全
18     .long FLAGS_ITEM_NAME     # 待补全
19     .long CHECKSUM_ITEM_NAME   # 待补全
20
21 .text # 进入代码段
22 .code32 # 32位代码
23
```

```

24 start: # 这个跟第一行的声明要一致
25     /*下面屏幕输出OK或其他字符序列*/
26     /*使用指令movl $0x12345678, 0xB8000*/ # 0x1234和0x5678各自输出1个字符
27     /*根据需要输出多个字符，也可以使用其他mov指令*/
28     /*可以根据需要使用nop指令隔开不同功能片段，也可以适当使用空行*/
29
30     movl $0x2f652f48, 0xB8000 /* H e */
31     movl $0x2f6c2f6c, 0xB8004 /* l l */
32     movl $0x2f202f6f, 0xB8008 /* o */
33     movl $0x2f6f2f57, 0xB800C /* w o */
34     movl $0x2f6c2f72, 0xB8010 /* r l */
35     movl $0x2f202f64, 0xB8014 /* d */
36     movl $0x2f422f50, 0xB8018 /* P B */
37     movl $0x2f322f32, 0xB801C /* 2 2 */
38     movl $0x2f352f30, 0xB8020 /* 0 5 */
39     movl $0x2f302f31, 0xB8024 /* 1 0 */
40     movl $0x2f312f33, 0xB8028 /* 3 1 */
41     movl $0x2f6c2f20, 0xB802C /* l */
42     movl $0x2f792f69, 0xB8030 /* i y */
43     movl $0x2f202f69, 0xB8034 /* i */
44     movl $0x2f202f20, 0xB8038 /* */
45     movl $0x2f202f20, 0xB803C /* */
46     movl $0x2f202f20, 0xB8040 /* */
47     movl $0x2f202f20, 0xB8044 /* */
48     movl $0x2f202f20, 0xB8048 /* */
49     movl $0x2f202f20, 0xB804C /* */
50
51     /*根据需要初始化串口*/
52     /*根据需要串口输出你的字符序列，详见前面串口编程简介*/
53     /* 初始化串口 */
54
55     movw $0x3F8, %dx
56     movb $0x50, %al /* P */
57     outb %al, %dx
58     movb $0x42, %al /* B */
59     outb %al, %dx
60     movb $0x32, %al /* 2 */
61     outb %al, %dx
62     movb $0x32, %al /* 2 */
63     outb %al, %dx
64     movb $0x30, %al /* 0 */
65     outb %al, %dx
66     movb $0x35, %al /* 5 */
67     outb %al, %dx
68     movb $0x31, %al /* 1 */
69     outb %al, %dx
70     movb $0x30, %al /* 0 */
71     outb %al, %dx
72     movb $0x33, %al /* 3 */
73     outb %al, %dx
74     movb $0x31, %al /* 1 */
75     outb %al, %dx
76     movb $0x20, %al /* 空格 */
77     outb %al, %dx
78     movb $0x6C, %al /* l */
79     outb %al, %dx
80     movb $0x69, %al /* i */
81     outb %al, %dx

```

```

82     movb $0x79, %a1 /* y */
83     outb %a1, %dx
84     movb $0x69, %a1 /* i */
85     outb %a1, %dx
86
87     /*实验结束，让计算机停机，方法：使用hlt指令，或者死循环*/
88 loop: hlt
89       jmp     loop

```

第三部分 编译过程进行说明

在src文件夹下输入：

```
1 | make && qemu-system-i386 -kernel multibootHeader.bin -serial stdio
```

即可运行

第四部分 运行结果

```

Machine  View
Hello World PB22051031 liyi

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCAF00+06F0AF00 CA00

Booting from ROM...

```

```

liyi@DESKTOP-6KORK05:~/OSLab$ cd lab1
liyi@DESKTOP-6KORK05:~/OSLab/lab1$ cd src
liyi@DESKTOP-6KORK05:~/OSLab/lab1/src$ make && qemu-system-i386 -kernel multibootHeader.bin -serial stdio
gcc -c -m32 --pipe -Wall -fasm -g -O1 -fno-stack-protector multibootHeader.S -o multibootHeader.o
ld -n -T multibootHeader.ld multibootHeader.o -o multibootHeader.bin
PB22051031 liyi

```

成功输出。